

Chapter

12

Enhanced Entity–Relationship Modeling

Chapter Objectives

In this chapter you will learn:

- The limitations of the basic concepts of the Entity–Relationship (ER) model and the requirements to represent more complex applications using additional data modeling concepts.
- The most useful additional data modeling concepts of the Enhanced Entity–Relationship (EER) model called specialization/generalization, aggregation, and composition.
- A diagrammatic technique for displaying specialization/generalization, aggregation, and composition in an EER diagram using the Unified Modeling Language (UML).

In Chapter 11 we discussed the basic concepts of the Entity–Relationship (ER) model. These basic concepts are normally adequate for building data models of traditional, administrative-based database systems such as stock control, product ordering, and customer invoicing. However, since the 1980s there has been a rapid increase in the development of many new database systems that have more demanding database requirements than those of the traditional applications. Examples of such database applications include Computer-Aided Design (CAD), Computer-Aided Manufacturing (CAM), Computer-Aided Software Engineering (CASE) tools, Office Information Systems (OIS) and Multimedia Systems, Digital Publishing, and Geographical Information Systems (GIS). The main features of these applications are described in Chapter 25. As the basic concepts of ER modeling are often not sufficient to represent the requirements of the newer, more complex applications, this stimulated the need to develop additional ‘semantic’ modeling concepts. Many different semantic data models have been proposed and some of the most important semantic concepts have been successfully incorporated into the original ER model. The ER model supported with additional semantic concepts is called the **Enhanced Entity–Relationship (EER)** model. In this chapter we describe three of the most important and useful additional concepts of the EER model, namely specialization/generalization, aggregation, and composition. We also illustrate how specialization/generalization, aggregation, and composition are represented in an EER diagram using the Unified Modeling Language (UML) (Booch *et al.*, 1998). In Chapter 11 we introduced UML and demonstrated how UML could be used to diagrammatically represent the basic concepts of the ER model.

Structure of this Chapter

In Section 12.1 we discuss the main concepts associated with specialization/generalization and illustrate how these concepts are represented in an EER diagram using the Unified Modeling Language (UML). We conclude this section with a worked example that demonstrates how to introduce specialization/generalization into an ER model using UML. In Section 12.2 we describe the concept of aggregation and in Section 12.3 the related concept of composition. We provide examples of aggregation and composition and show how these concepts can be represented in an EER diagram using UML.

12.1 Specialization/Generalization

The concept of specialization/generalization is associated with special types of entities known as **superclasses** and **subclasses**, and the process of **attribute inheritance**. We begin this section by defining what superclasses and subclasses are and by examining superclass/subclass relationships. We describe the process of attribute inheritance and contrast the process of specialization with the process of generalization. We then describe the two main types of constraints on superclass/subclass relationships called participation and disjoint constraints. We show how to represent specialization/generalization in an Enhanced Entity–Relationship (EER) diagram using UML. We conclude this section with a worked example of how specialization/generalization may be introduced into the Entity–Relationship (ER) model of the Branch user views of the *DreamHome* case study described in Appendix A and shown in Figure 11.1.

12.1.1 Superclasses and Subclasses

As we discussed in Chapter 11, an entity type represents a set of entities of the same type such as Staff, Branch, and PropertyForRent. We can also form entity types into a hierarchy containing superclasses and subclasses.

Superclass An entity type that includes one or more distinct subgroupings of its occurrences, which require to be represented in a data model.

Subclass A distinct subgrouping of occurrences of an entity type, which require to be represented in a data model.

Entity types that have distinct subclasses are called superclasses. For example, the entities that are members of the Staff entity type may be classified as Manager, SalesPersonnel, and Secretary. In other words, the Staff entity is referred to as the **superclass** of the Manager, SalesPersonnel, and Secretary **subclasses**. The relationship between a superclass and any

one of its subclasses is called a superclass/subclass relationship. For example, Staff/Manager has a superclass/subclass relationship.

Superclass/Subclass Relationships

12.1.2

Each member of a subclass is also a member of the superclass. In other words, the entity in the subclass is the same entity in the superclass, but has a distinct role. The relationship between a superclass and a subclass is one-to-one (1:1) and is called a superclass/subclass relationship (see Section 11.6.1). Some superclasses may contain overlapping subclasses, as illustrated by a member of staff who is both a Manager and a member of Sales Personnel. In this example, Manager and SalesPersonnel are overlapping subclasses of the Staff superclass. On the other hand, not every member of a superclass need be a member of a subclass; for example, members of staff without a distinct job role such as a Manager or a member of Sales Personnel.

We can use superclasses and subclasses to avoid describing different types of staff with possibly different attributes within a single entity. For example, Sales Personnel may have special attributes such as salesArea and carAllowance. If all staff attributes and those specific to particular jobs are described by a single Staff entity, this may result in a lot of nulls for the job-specific attributes. Clearly, Sales Personnel have common attributes with other staff, such as staffNo, name, position, and salary. However, it is the unshared attributes that cause problems when we try to represent all members of staff within a single entity. We can also show relationships that are only associated with particular types of staff (subclasses) and not with staff, in general. For example, Sales Personnel may have distinct relationships that are not appropriate for all staff, such as SalesPersonnel Uses Car.

To illustrate these points, consider the relation called AllStaff shown in Figure 12.1. This relation holds the details of all members of staff no matter what position they hold. A consequence of holding all staff details in one relation is that while the attributes appropriate to all staff are filled (namely, staffNo, name, position, and salary), those that are only applicable

staffNo	name	position	salary	mgrStartDate	bonus	salesArea	carAllowance	typingSpeed
SL21	John White	Manager	30000	01/02/95	2000			
SG37	Ann Beech	Assistant	12000					
SG66	Mary Martinez	Sales Manager	27000			SA1A	5000	
SA9	Mary Howe	Assistant	9000					
SL89	Stuart Stern	Secretary	8500					100
SL31	Robert Chin	Snr Sales Asst	17000			SA2B	3700	
SG5	Susan Brand	Manager	24000	01/06/91	2350			

Figure 12.1
The AllStaff relation holding details of all staff.

to particular job roles are only partially filled. For example, the attributes associated with the Manager (mgrStartDate and bonus), SalesPersonnel (salesArea and carAllowance), and Secretary (typingSpeed) subclasses have values for those members in these subclasses. In other words, the attributes associated with the Manager, SalesPersonnel, and Secretary subclasses are empty for those members of staff not in these subclasses.

There are two important reasons for introducing the concepts of superclasses and subclasses into an ER model. Firstly, it avoids describing similar concepts more than once, thereby saving time for the designer and making the ER diagram more readable. Secondly, it adds more semantic information to the design in a form that is familiar to many people. For example, the assertions that ‘Manager IS-A member of staff’ and ‘flat IS-A type of property’, communicates significant semantic content in a concise form.

12.1.3 Attribute Inheritance

As mentioned above, an entity in a subclass represents the same ‘real world’ object as in the superclass, and may possess subclass-specific attributes, as well as those associated with the superclass. For example, a member of the SalesPersonnel subclass *inherits* all the attributes of the Staff superclass such as staffNo, name, position, and salary together with those specifically associated with the SalesPersonnel subclass such as salesArea and carAllowance.

A subclass is an entity in its own right and so it may also have one or more subclasses. An entity and its subclasses and their subclasses, and so on, is called a **type hierarchy**. Type hierarchies are known by a variety of names including: **specialization hierarchy** (for example, Manager is a specialization of Staff), **generalization hierarchy** (for example, Staff is a generalization of Manager), and **IS-A hierarchy** (for example, Manager IS-A (member of) Staff). We describe the process of specialization and generalization in the following sections.

A subclass with more than one superclass is called a **shared subclass**. In other words, a member of a shared subclass must be a member of the associated superclasses. As a consequence, the attributes of the superclasses are inherited by the shared subclass, which may also have its own additional attributes. This process is referred to as **multiple inheritance**.

12.1.4 Specialization Process

Specialization The process of maximizing the differences between members of an entity by identifying their distinguishing characteristics.

Specialization is a top-down approach to defining a set of superclasses and their related subclasses. The set of subclasses is defined on the basis of some distinguishing characteristics of the entities in the superclass. When we identify a set of subclasses of an entity type, we then associate attributes specific to each subclass (where necessary), and also identify any relationships between each subclass and other entity types or subclasses (where necessary). For example, consider a model where all members of staff are

represented as an entity called *Staff*. If we apply the process of specialization on the *Staff* entity, we attempt to identify differences between members of this entity such as members with distinctive attributes and/or relationships. As described earlier, staff with the job roles of *Manager*, *Sales Personnel*, and *Secretary* have distinctive attributes and therefore we identify *Manager*, *SalesPersonnel*, and *Secretary* as subclasses of a specialized *Staff* superclass.

Generalization Process

12.1.5

Generalization The process of minimizing the differences between entities by identifying their common characteristics.

The process of generalization is a bottom-up approach, which results in the identification of a generalized superclass from the original entity types. For example, consider a model where *Manager*, *SalesPersonnel*, and *Secretary* are represented as distinct entity types. If we apply the process of generalization on these entities, we attempt to identify similarities between them such as common attributes and relationships. As stated earlier, these entities share attributes common to all staff, and therefore we identify *Manager*, *SalesPersonnel*, and *Secretary* as subclasses of a generalized *Staff* superclass.

As the process of generalization can be viewed as the reverse of the specialization process, we refer to this modeling concept as ‘specialization/generalization’.

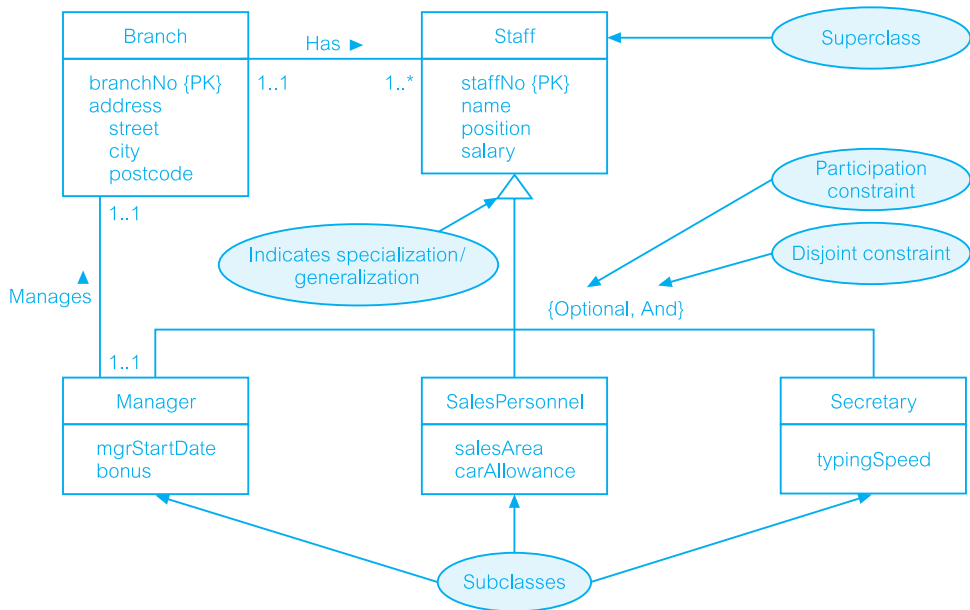
Diagrammatic representation of specialization/generalization

UML has a special notation for representing specialization/generalization. For example, consider the specialization/generalization of the *Staff* entity into subclasses that represent job roles. The *Staff* superclass and the *Manager*, *SalesPersonnel*, and *Secretary* subclasses can be represented in an Enhanced Entity–Relationship (EER) diagram as illustrated in Figure 12.2. Note that the *Staff* superclass and the subclasses, being entities, are represented as rectangles. The subclasses are attached by lines to a triangle that points toward the superclass. The label below the specialization/generalization triangle, shown as {Optional, And}, describes the constraints on the relationship between the superclass and its subclasses. These constraints are discussed in more detail in Section 12.1.6.

Attributes that are specific to a given subclass are listed in the lower section of the rectangle representing that subclass. For example, *salesArea* and *carAllowance* attributes are only associated with the *SalesPersonnel* subclass, and are not applicable to the *Manager* or *Secretary* subclasses. Similarly, we show attributes that are specific to the *Manager* (*mgrStartDate* and *bonus*) and *Secretary* (*typingSpeed*) subclasses.

Attributes that are common to all subclasses are listed in the lower section of the rectangle representing the superclass. For example, *staffNo*, *name*, *position*, and *salary* attributes are common to all members of staff and are associated with the *Staff* superclass. Note that we can also show relationships that are only applicable to specific subclasses. For example, in Figure 12.2, the *Manager* subclass is related to the *Branch* entity through the

Figure 12.2
Specialization/
generalization of
the Staff entity
into subclasses
representing
job roles.



Manages relationship, whereas the Staff superclass is related to the Branch entity through the *Has* relationship.

We may have several specializations of the same entity based on different distinguishing characteristics. For example, another specialization of the Staff entity may produce the subclasses *FullTimePermanent* and *PartTimeTemporary*, which distinguishes between the types of employment contract for members of staff. The specialization of the Staff entity type into job role and contract of employment subclasses is shown in Figure 12.3. In this figure, we show attributes that are specific to the *FullTimePermanent* (salaryScale and holidayAllowance) and *PartTimeTemporary* (hourlyRate) subclasses.

As described earlier, a superclass and its subclasses and their subclasses, and so on, is called a type hierarchy. An example of a type hierarchy is shown in Figure 12.4, where the job roles specialization/generalization shown in Figure 12.2 are expanded to show a shared subclass called *SalesManager* and the subclass called *Secretary* with its own subclass called *AssistantSecretary*. In other words, a member of the *SalesManager* shared subclass must be a member of the *SalesPersonnel* and *Manager* subclasses as well as the Staff superclass. As a consequence, the attributes of the Staff superclass (staffNo, name, position, and salary), and the attributes of the subclasses *SalesPersonnel* (salesArea and carAllowance) and *Manager* (mgrStartDate and bonus) are inherited by the *SalesManager* subclass, which also has its own additional attribute called salesTarget.

AssistantSecretary is a subclass of *Secretary*, which is a subclass of Staff. This means that a member of the *AssistantSecretary* subclass must be a member of the *Secretary* subclass and the Staff superclass. As a consequence, the attributes of the Staff superclass (staffNo, name, position, and salary) and the attribute of the *Secretary* subclass (typingSpeed) are inherited by the *AssistantSecretary* subclass, which also has its own additional attribute called startDate.

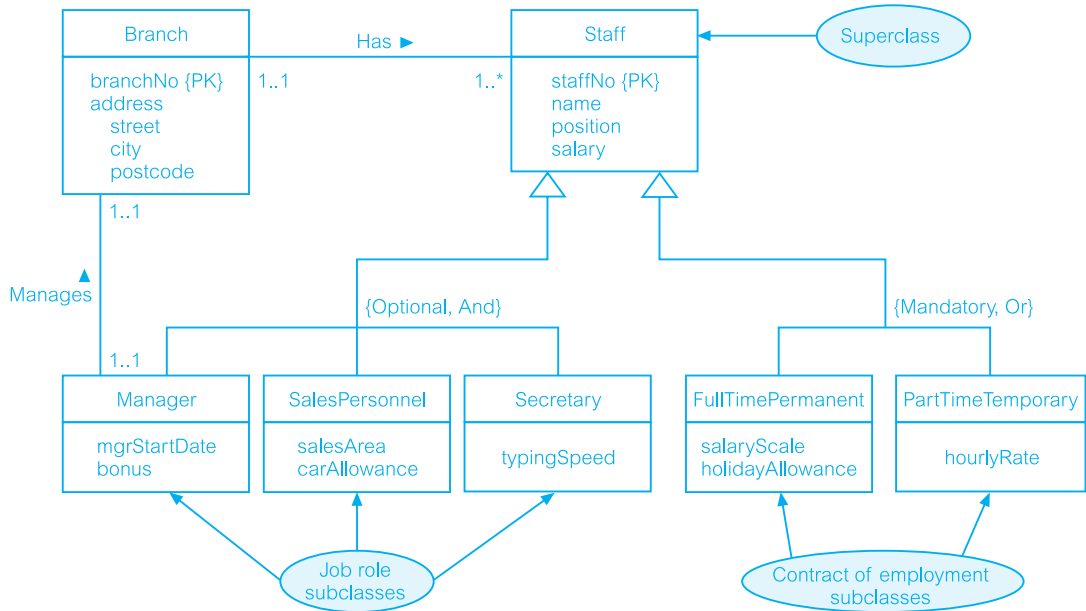


Figure 12.3 Specialization/generalization of the Staff entity into subclasses representing job roles and contracts of employment.

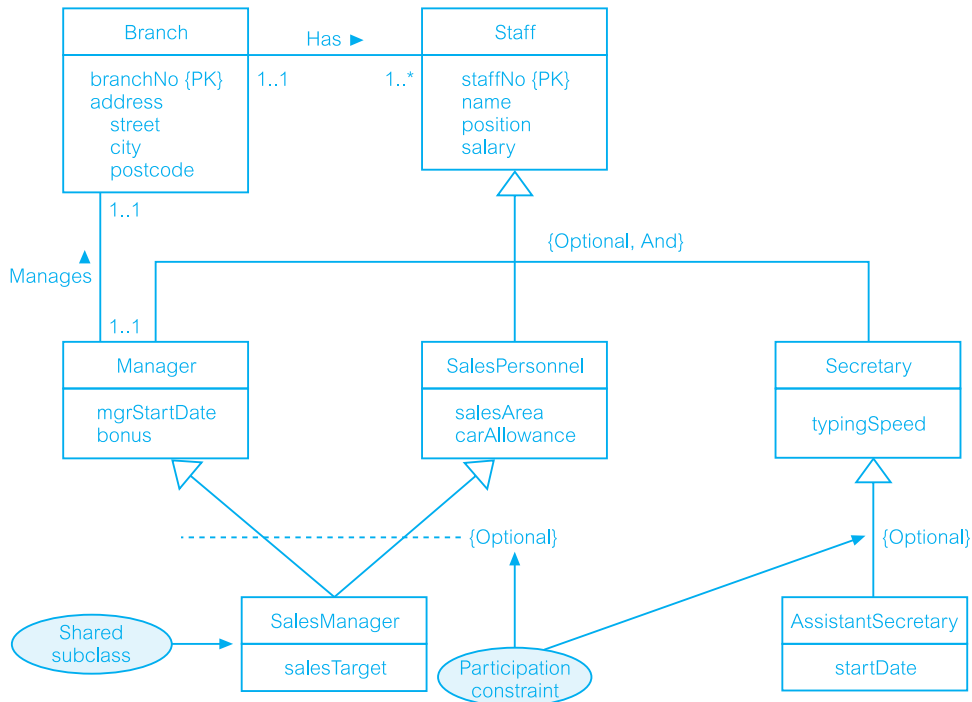


Figure 12.4 Specialization/generalization of the Staff entity into job roles including a shared subclass called SalesManager and a subclass called Secretary with its own subclass called AssistantSecretary.

12.1.6 Constraints on Specialization/Generalization

There are two constraints that may apply to a specialization/generalization called **participation constraints** and **disjoint constraints**.

Participation constraints

Participation constraint	Determines whether every member in the superclass must participate as a member of a subclass.
---------------------------------	---

A participation constraint may be **mandatory** or **optional**. A superclass/subclass relationship with mandatory participation specifies that every member in the superclass must also be a member of a subclass. To represent mandatory participation, ‘Mandatory’ is placed in curly brackets below the triangle that points towards the superclass. For example, in Figure 12.3 the contract of employment specialization/generalization is mandatory participation, which means that every member of staff must have a contract of employment.

A superclass/subclass relationship with optional participation specifies that a member of a superclass need not belong to any of its subclasses. To represent optional participation, ‘Optional’ is placed in curly brackets below the triangle that points towards the superclass. For example, in Figure 12.3 the job role specialization/generalization has optional participation, which means that a member of staff need not have an additional job role such as a Manager, Sales Personnel, or Secretary.

Disjoint constraints

Disjoint constraint	Describes the relationship between members of the subclasses and indicates whether it is possible for a member of a superclass to be a member of one, or more than one, subclass.
----------------------------	---

The disjoint constraint only applies when a superclass has more than one subclass. If the subclasses are **disjoint**, then an entity occurrence can be a member of only one of the subclasses. To represent a disjoint superclass/subclass relationship, ‘Or’ is placed next to the participation constraint within the curly brackets. For example, in Figure 12.3 the subclasses of the contract of employment specialization/generalization is disjoint, which means that a member of staff must have a full-time permanent *or* a part-time temporary contract, but not both.

If subclasses of a specialization/generalization are not disjoint (called **nondisjoint**), then an entity occurrence may be a member of more than one subclass. To represent a nondisjoint superclass/subclass relationship, ‘And’ is placed next to the participation constraint within the curly brackets. For example, in Figure 12.3 the job role specialization/generalization is nondisjoint, which means that an entity occurrence can be a member of both the Manager, SalesPersonnel, and Secretary subclasses. This is confirmed by the presence of the shared subclass called SalesManager shown in Figure 12.4. Note that it is not necessary to include the disjoint constraint for hierarchies that have a single subclass

at a given level and for this reason only the participation constraint is shown for the SalesManager and AssistantSecretary subclasses of Figure 12.4.

The disjoint and participation constraints of specialization and generalization are distinct, giving rise to four categories: ‘mandatory and disjoint’, ‘optional and disjoint’, ‘mandatory and nondisjoint’, and ‘optional and nondisjoint’.

Worked Example of using Specialization/Generalization to Model the Branch View of DreamHome Case Study

12.1.7

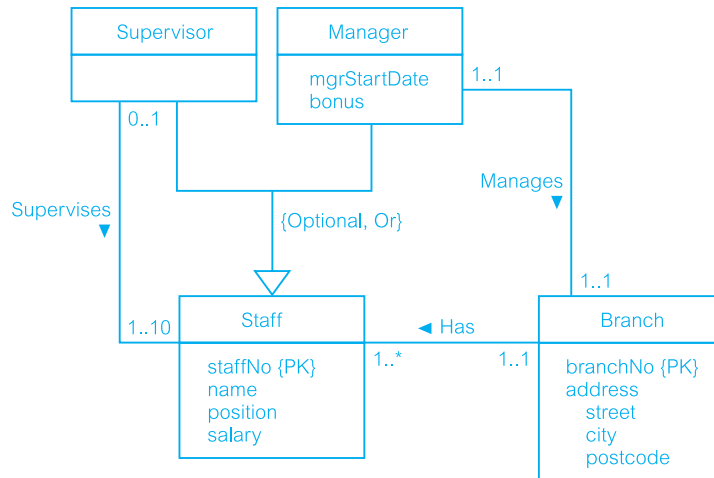
The database design methodology described in this book includes the use of specialization/generalization as an optional step (Step 1.6) in building an EER model. The choice to use this step is dependent on the complexity of the enterprise (or part of the enterprise) being modeled and whether using the additional concepts of the EER model will help the process of database design.

In Chapter 11 we described the basic concepts necessary to build an ER model to represent the Branch user views of the *DreamHome* case study. This model was shown as an ER diagram in Figure 11.1. In this section, we show how specialization/generalization may be used to convert the ER model of the Branch user views into an EER model.

As a starting point, we first consider the entities shown in Figure 11.1. We examine the attributes and relationships associated with each entity to identify any similarities or differences between the entities. In the Branch user views’ requirements specification there are several instances where there is the potential to use specialization/generalization as discussed below.

- (a) For example, consider the Staff entity in Figure 11.1, which represents all members of staff. However, in the data requirements specification for the Branch user views of the *DreamHome* case study given in Appendix A, there are two key job roles mentioned namely Manager and Supervisor. We have three options as to how we may best model members of staff. The first option is to represent all members of staff as a generalized Staff entity (as in Figure 11.1), the second option is to create three distinct entities Staff, Manager, and Supervisor, and the third option is to represent the Manager and Supervisor entities as subclasses of a Staff superclass. The option we select is based on the commonality of attributes and relationships associated with each entity. For example, all attributes of the Staff entity are represented in the Manager and Supervisor entities, including the same primary key, namely staffNo. Furthermore, the Supervisor entity does not have any additional attributes representing this job role. On the other hand, the Manager entity has two additional attributes, namely mgrStartDate and bonus. In addition, both the Manager and Supervisor entities are associated with distinct relationships, namely Manager *Manages* Branch and Supervisor *Supervises* Staff. Based on this information, we select the third option and create Manager and Supervisor subclasses of the Staff superclass, as shown in Figure 12.5. Note that in this EER diagram, the subclasses are shown above the superclass. The relative positioning of the subclasses and superclass is not significant, however; what is important is that the specialization/generalization triangle points toward the superclass.

Figure 12.5
Staff superclass
with Supervisor
and Manager
subclasses.

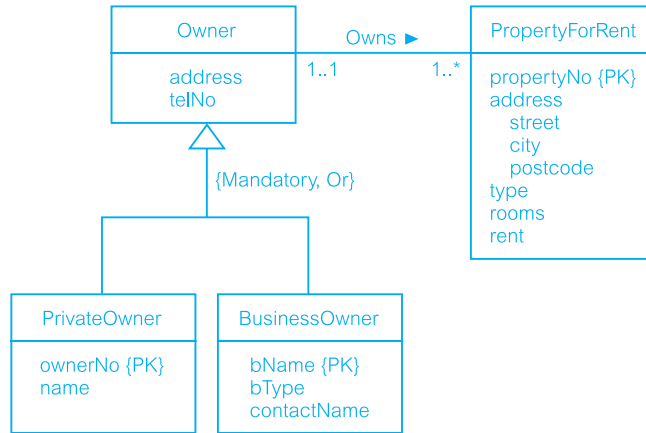


The specialization/generalization of the Staff entity is optional and disjoint (shown as {Optional, Or}), as not all members of staff are Managers or Supervisors, and in addition a single member of staff cannot be both a Manager and a Supervisor. This representation is particularly useful for displaying the shared attributes associated with these subclasses and the Staff superclass and also the distinct relationships associated with each subclass, namely Manager *Manages* Branch and Supervisor *Supervises* Staff.

- (b) Next, consider for specialization/generalization the relationship between owners of property. The data requirements specification for the Branch user views describes two types of owner, namely PrivateOwner and BusinessOwner as shown in Figure 11.1. Again, we have three options as to how we may best model owners of property. The first option is to leave PrivateOwner and BusinessOwner as two distinct entities (as shown in Figure 11.1), the second option is to represent both types of owner as a generalized Owner entity, and the third option is to represent the PrivateOwner and BusinessOwner entities as subclasses of an Owner superclass. Before we are able to reach a decision we first examine the attributes and relationships associated with these entities. PrivateOwner and BusinessOwner entities share common attributes, namely address and telNo and have a similar relationship with property for rent (namely PrivateOwner *POwns* PropertyForRent and BusinessOwner *BOwns* PropertyForRent). However, both types of owner also have different attributes; for example, PrivateOwner has distinct attributes ownerNo and name, and BusinessOwner has distinct attributes bName, bType, and contactName. In this case, we create a superclass called Owner, with PrivateOwner and BusinessOwner as subclasses as shown in Figure 12.6.

The specialization/generalization of the Owner entity is mandatory and disjoint (shown as {Mandatory, Or}), as an owner must be either a private owner *or* a business owner, but cannot be both. Note that we choose to relate the Owner superclass to the PropertyForRent entity using the relationship called Owns.

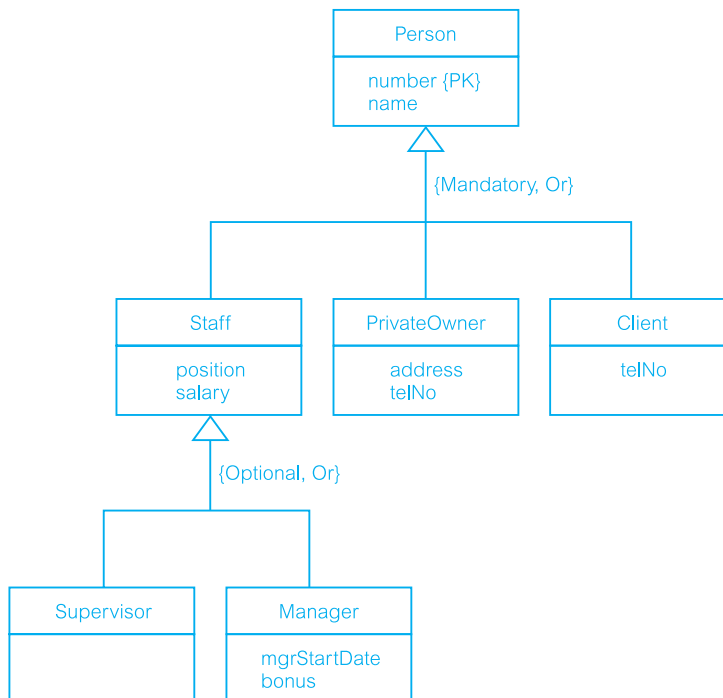
The examples of specialization/generalization described above are relatively straightforward. However, the specialization/generalization process can be taken further as illustrated in the following example.

**Figure 12.6**

Owner superclass with PrivateOwner and BusinessOwner subclasses.

- (c) There are several persons with common characteristics described in the data requirements specification for the Branch user views of the *DreamHome* case study. For example, members of staff, private property owners, and clients all have number and name attributes. We could create a Person superclass with Staff (including Manager and Supervisor subclasses), PrivateOwner, and Client as subclasses, as shown in Figure 12.7.

We now consider to what extent we wish to use specialization/generalization to represent the Branch user views of the *DreamHome* case study. We decide to use the specialization/generalization examples described in (a) and (b) above but not (c), as shown in Figure 12.8. To simplify the EER diagram only attributes associated with primary keys or

**Figure 12.7**

Person superclass with Staff (including Supervisor and Manager subclasses), PrivateOwner, and Client subclasses.

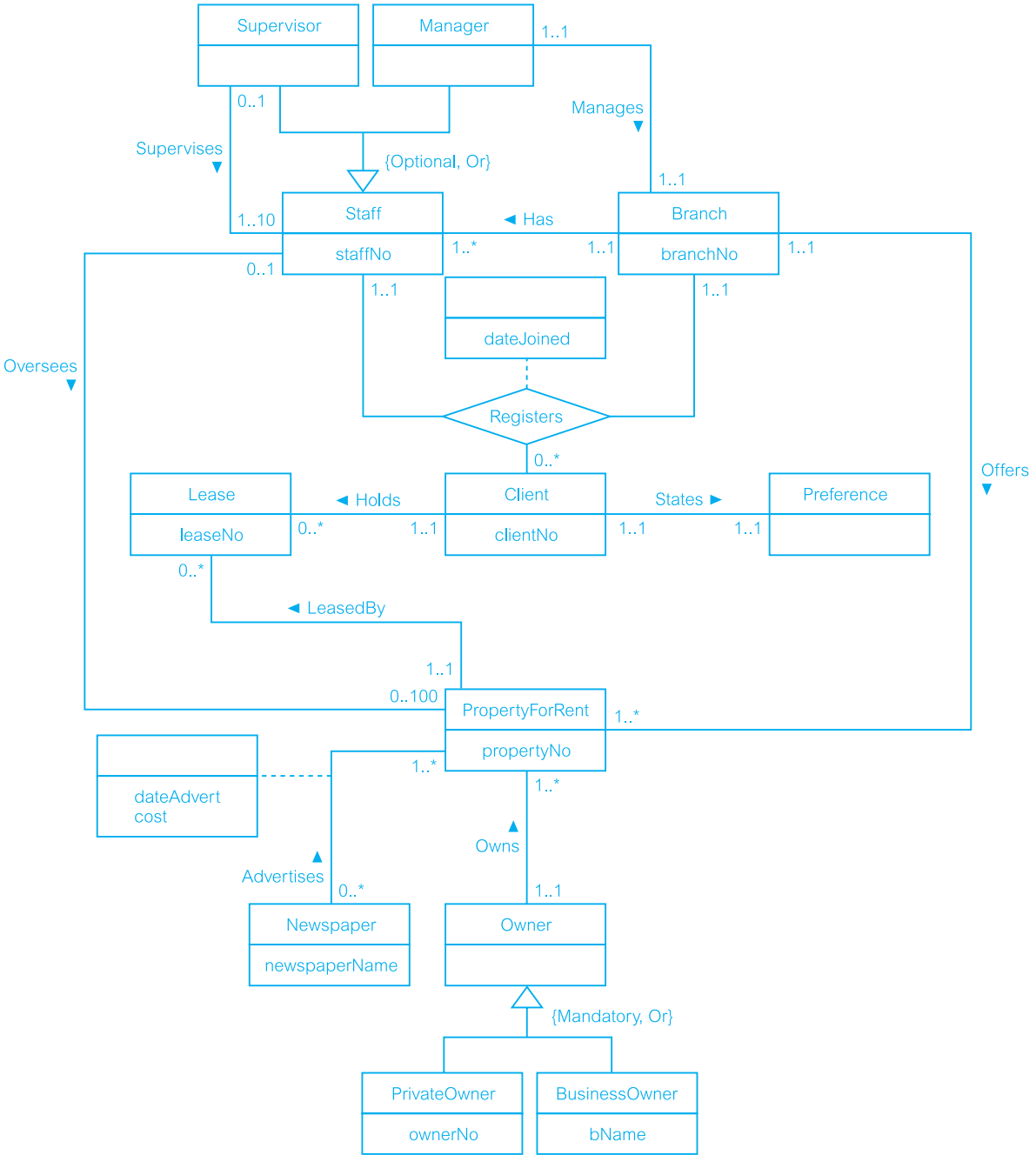


Figure 12.8 An Enhanced Entity–Relationship (EER) model of the Branch user views of *DreamHome* with specialization/generalization.

relationships are shown. We leave out the representation shown in Figure 12.7 from the final EER model because the use of specialization/generalization in this case places too much emphasis on the relationship between entities that are persons rather than emphasizing the relationship between these entities and some of the core entities such as Branch and PropertyForRent.

The option to use specialization/generalization, and to what extent, is a subjective decision. In fact, the use of specialization/generalization is presented as an optional step in our methodology for conceptual database design discussed in Chapter 15, Step 1.6.

As described in Section 2.3, the purpose of a data model is to provide the concepts and notations that allow database designers and end-users to unambiguously and accurately communicate their understanding of the enterprise data. Therefore, if we keep these goals in mind, we should only use the additional concepts of specialization/generalization when the enterprise data is too complex to easily represent using only the basic concepts of the ER model.

At this stage we may consider whether the introduction of specialization/generalization to represent the Branch user views of *DreamHome* is a good idea. In other words, is the requirement specification for the Branch user views better represented as the ER model shown in Figure 11.1 or as the EER model shown in Figure 12.8? We leave this for the reader to consider.

Aggregation

12.2

Aggregation Represents a ‘has-a’ or ‘is-part-of’ relationship between entity types, where one represents the ‘whole’ and the other the ‘part’.

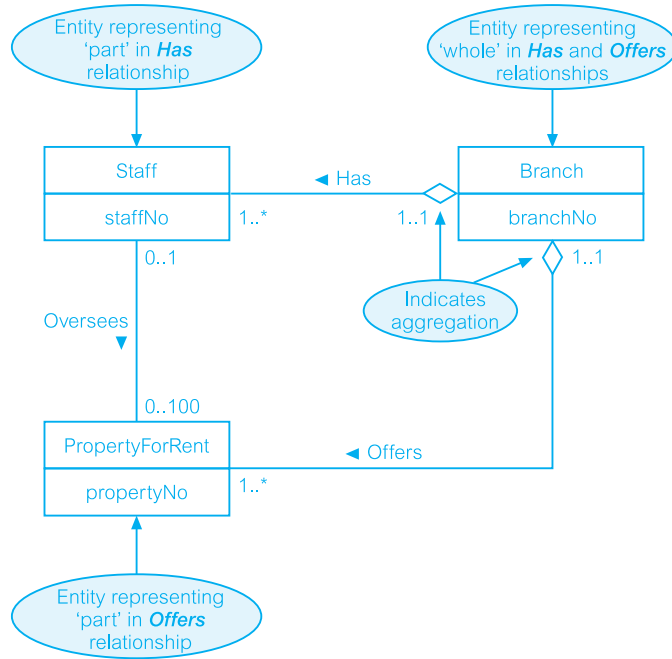
A relationship represents an association between two entity types that are conceptually at the same level. Sometimes we want to model a ‘has-a’ or ‘is-part-of’ relationship, in which one entity represents a larger entity (the ‘whole’), consisting of smaller entities (the ‘parts’). This special kind of relationship is called an **aggregation** (Booch *et al.*, 1998). Aggregation does not change the meaning of navigation across the relationship between the whole and its parts, nor does it link the lifetimes of the whole and its parts. An example of an aggregation is the *Has* relationship, which relates the Branch entity (the ‘whole’) to the Staff entity (the ‘part’).

Diagrammatic representation of aggregation

UML represents aggregation by placing an open diamond shape at one end of the relationship line, next to the entity that represents the ‘whole’. In Figure 12.9, we redraw part of the EER diagram shown in Figure 12.8 to demonstrate aggregation. This EER diagram displays two examples of aggregation, namely Branch *Has* Staff and Branch *Offers* PropertyForRent. In both relationships, the Branch entity represents the ‘whole’ and therefore the open diamond shape is placed beside this entity.

Figure 12.9

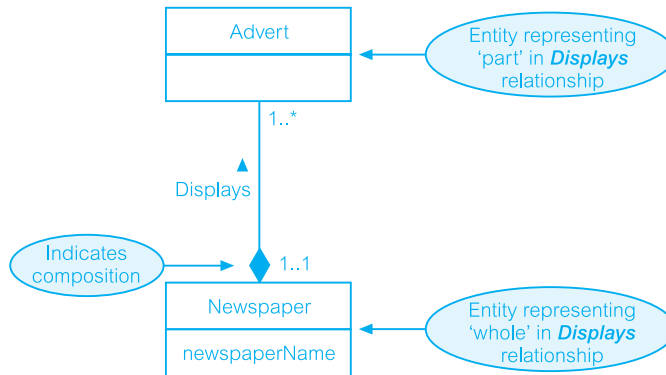
Examples of aggregation:
Branch *Has* Staff
and Branch *Offers*
PropertyForRent.



12.3 Composition

Composition A specific form of aggregation that represents an association between entities, where there is a strong ownership and coincidental lifetime between the ‘whole’ and the ‘part’.

Aggregation is entirely conceptual and does nothing more than distinguish a ‘whole’ from a ‘part’. However, there is a variation of aggregation called **composition** that represents a strong ownership and coincidental lifetime between the ‘whole’ and the ‘part’ (Booch *et al.*, 1998). In a composite, the ‘whole’ is responsible for the disposition of the ‘parts’, which means that the composition must manage the creation and destruction of its ‘parts’. In other words, an object may only be part of one composite at a time. There are no examples of composition in Figure 12.8. For the purposes of discussion, consider an example of a composition, namely the *Displays* relationship, which relates the Newspaper entity to the Advert entity. As a composition, this emphasizes the fact that an Advert entity (the ‘part’) belongs to exactly one Newspaper entity (the ‘whole’). This is in contrast to aggregation, in which a part may be shared by many wholes. For example, a Staff entity may be ‘a part of’ one or more Branches entities.

**Figure 12.10**

An example of composition:
Newspaper
Displays Advert.

Diagrammatic representation of composition

UML represents composition by placing a filled-in diamond shape at one end of the relationship line next to the entity that represents the 'whole' in the relationship. For example, to represent the *Newspaper Displays Advert* composition, the filled-in diamond shape is placed next to the *Newspaper* entity, which is the 'whole' in this relationship, as shown in Figure 12.10.

As discussed with specialization/generalization, the options to use aggregation and composition, and to what extent, are again subjective decisions. Aggregation and composition should only be used when there is a requirement to emphasize special relationships between entity types such as 'has-a' or 'is-part-of', which has implications on the creation, update, and deletion of these closely related entities. We discuss how to represent such constraints between entity types in our methodology for logical database design in Chapter 16, Step 2.4.

If we remember that the major aim of a data model is to unambiguously and accurately communicate an understanding of the enterprise data. We should only use the additional concepts of aggregation and composition when the enterprise data is too complex to easily represent using only the basic concepts of the ER model.

Chapter Summary

- A **superclass** is an entity type that includes one or more distinct subgroupings of its occurrences, which require to be represented in a data model. A **subclass** is a distinct subgrouping of occurrences of an entity type, which require to be represented in a data model.
- **Specialization** is the process of maximizing the differences between members of an entity by identifying their distinguishing features.
- **Generalization** is the process of minimizing the differences between entities by identifying their common features.
- There are two constraints that may apply to a specialization/generalization called **participation constraints** and **disjoint constraints**.

- A **participation constraint** determines whether every member in the superclass must participate as a member of a subclass.
- A **disjoint constraint** describes the relationship between members of the subclasses and indicates whether it is possible for a member of a superclass to be a member of one, or more than one, subclass.
- **Aggregation** represents a ‘has-a’ or ‘is-part-of’ relationship between entity types, where one represents the ‘whole’ and the other the ‘part’.
- **Composition** is a specific form of aggregation that represents an association between entities, where there is a strong ownership and coincidental lifetime between the ‘whole’ and the ‘part’.

Review Questions

- | | |
|--|--|
| 12.1 Describe what a superclass and a subclass represent. | and how this concept relates to multiple inheritance. |
| 12.2 Describe the relationship between a superclass and its subclass. | 12.6 Describe and contrast the process of specialization with the process of generalization. |
| 12.3 Describe and illustrate using an example the process of attribute inheritance. | 12.7 Describe the two main constraints that apply to a specialization/generalization relationship. |
| 12.4 What are the main reasons for introducing the concepts of superclasses and subclasses into an ER model? | 12.8 Describe and contrast the concepts of aggregation and composition and provide an example of each. |
| 12.5 Describe what a shared subclass represents | |

Exercises

- 12.9 Consider whether it is appropriate to introduce the enhanced concepts of specialization/generalization, aggregation, and/or composition for the case studies described in Appendix B.
- 12.10 Consider whether it is appropriate to introduce the enhanced concepts of specialization/generalization, aggregation, and/or composition into the ER model for the case study described in Exercise 11.12. If appropriate, redraw the ER diagram as an EER diagram with the additional enhanced concepts.
-